

Lab 07 · El talonario de folios

Curso de Spring Boot · Servicio de Impuestos Internos · 2026

75 minutos · Spring Boot 4.1.0 · Java 25 (Temurin) · PostgreSQL 16 embebido

Resumen

Que un código correcto para un usuario puede ser incorrecto para veinte a la vez. Se emiten veinte folios simultáneos: **sin turno sobreviven 10 y la base rechaza 11; con un turno con nombre, 21 de 21.**

Índice

1. Antes de empezar	2
1.1. Qué vas a lograr	2
1.2. Qué necesitas tener listo	2
1.3. Cómo copiar el código de esta guía	2
1.4. La puesta a punto	3
2. El caso	3
2.1. El talonario, que es la metáfora de este laboratorio	3
3. Los pasos	3
3.1. Paso 1 · De uno en uno, y todo va bien	3
3.2. Paso 2 · El crimen: veinte a la vez	4
3.3. Paso 3 · La trampa: <code>synchronized</code> no sirve	5
3.4. Paso 4 · El turno con nombre	6
3.5. Paso 5 · El cinturón	8
4. Lo que aprendiste	9
5. Para profundizar	9
6. Antes de cerrar	9

1 Antes de empezar

1.1 Qué vas a lograr

Todo lo que has escrito hasta hoy funciona cuando lo usa **una persona**. Hoy vas a ver qué pasa cuando lo usan **veinte a la vez**, y no es lo que parece.

Vas a escribir un emisor de folios correlativos, vas a verlo funcionar perfectamente en secuencial, y después vas a lanzarle veinte peticiones simultáneas y a contar cuántas sobreviven. Vas a aprender por qué `synchronized` **no** lo arregla, y a poner las dos defensas que sí: un turno con nombre en la base y una restricción que actúa de cinturón.

1.2 Qué necesitas tener listo

Requisito	Cómo lo compruebas	Qué tiene que salir
Los labs 04 y 05 hechos	Sabes qué es una transacción	Imprescindible
Estar en la carpeta del lab	<code>cd labs/lab-07-concurrencia/practica</code>	El cd no da error

1.3 Cómo copiar el código de esta guía

Al copiar de un PDF se pierden los espacios del principio de línea, y a veces una línea larga se parte en dos. Con Java no importa. El código completo está en `labs/lab-07-concurrencia/solucion/`.

1.4 La puesta a punto

```
cd labs/lab-07-concurrencia/practica
./mvnw spring-boot:run
```

Escucha en el **8091** y su PostgreSQL en el **55438**. **Párala con Ctrl+C**.

2 El caso

La DGT emite folios: **un número correlativo por año**, sin saltos y sin repetidos. 2026-0001, 2026-0002, y así. Un folio repetido es un problema legal, no un problema técnico.

2.1 El talonario, que es la metáfora de este laboratorio

Un talonario en un cajón, y veinte funcionarios.

El procedimiento es obvio: **miras cuál fue el último número**, le sumas uno, y **arrancas la hoja**. Con un funcionario funciona siempre.

Ahora pon veinte funcionarios delante del mismo cajón. Los veinte miran **a la vez**, los veinte ven el mismo último número, y los veinte arrancan la hoja siguiente. **Diecinueve se llevan el mismo folio**.

El fallo no está en el procedimiento: está en que **entre mirar y arrancar hay un hueco**, y en ese hueco caben los demás.

La solución del mundo real es la misma que la del software: **el que va a arrancar una hoja cierra el cajón con llave**. Los demás esperan a que la devuelva. Eso es el bloqueo pesimista, y es el paso 3.

Y el cinturón: aunque alguien se salte el procedimiento, **el propio talonario no admite dos hojas con el mismo número**. Eso es la restricción UNIQUE de la base, y es el paso 4.

3 Los pasos

3.1 Paso 1 · De uno en uno, y todo va bien

Qué vamos a hacer

Emitir once folios **en secuencia** y comprobar que salen perfectos.

Para entenderlo mejor

Un funcionario, un talonario. Mira, suma, arranca. Once veces. No hay forma de que falle.

El problema

Este paso existe para **quitar una sospecha**. Cuando en el paso 2 todo se rompa, la primera reacción es pensar que el código está mal escrito. No lo está: es el mismo código, y aquí funciona.

Se pega

El servicio, en `practica/src/main/java/cl/dgt/concurrencia/services/EmisorDeFolios.java`:

```
@Transactional
public Folio emitirIngenuo(int anio) {
    int ultimo = folios.maxNumeroDe(anio).orElse(0);
    return folios.save(new Folio(anio, ultimo + 1));
}
```

Léelo despacio, porque es el código del crimen y no tiene nada malo a la vista: busca el máximo, suma uno, guarda. Es lo que escribiría cualquiera.

Lo que vas a ver

```
=== 1 · DE UNO EN UNO · secuencial ===
año 2026 reiniciado: solo el folio 2026-0001
folios en la tabla : 11
números distintos : 11
REPETIDOS          : ninguno
rechazados por la base : 0
emitidos: [2026-0001, 2026-0002, ... 2026-0011]
```

Once folios, once números distintos, cero repetidos. El código es correcto.

Vas bien si...

folios en la tabla y números distintos son el mismo número, y REPETIDOS dice ninguno.

Si te atascas

1 · EL PUERTO 55438 YA ESTA OCUPADO O ESTE MISMO PROYECTO YA ESTA CORRIENDO

Los dos candados del Lab 04, con los números de este lab:

```
lsof -ti:55438 | xargs kill -9
```

2 · Salen más de 11 folios.

La tabla trae folios de una corrida anterior. La demo reinicia el año al empezar; si tocaste el código de reinicio, borra .datos-pg/ y arranca de nuevo.

3.2 Paso 2 · El crimen: veinte a la vez

Qué vamos a hacer

Lanzar **veinte emisiones simultáneas** con el mismo código del paso 1, y contar los daños.

Para entenderlo mejor

Los veinte funcionarios delante del cajón. Los veinte miran, los veinte ven lo mismo, los veinte arrancan.

El problema

Entre select max(numero) y el insert hay un hueco de tiempo. Dentro de ese hueco, **otras diecinueve transacciones hacen exactamente lo mismo**. Todas leen el mismo máximo. Todas calculan el mismo siguiente.

Y esto no es una rareza de laboratorio: **es lo que pasa en cuanto dos personas usan la aplicación al mismo tiempo**. La diferencia entre este lab y producción es que aquí lo estás provocando a propósito.

Lo que vas a ver

```
=== 2 · EL CRIMEN · 20 emisiones a la vez, sin protección ===
año 2026 reiniciado: solo el folio 2026-0001
Detail: Key (anio, numero)=(2026, 4) already exists.
Detail: Key (anio, numero)=(2026, 3) already exists.
Detail: Key (anio, numero)=(2026, 2) already exists.
...
folios en la tabla : 10
números distintos : 10
REPETIDOS          : ninguno
rechazados por la base : 11
```

Lee los tres números juntos, porque cuentan una historia:

- Se pidieron **20** folios.
- Sobrevivieron **10**.
- La base **rechazó 11**, y dice cuáles: Key (anio, numero)=(2026, 4) already exists.

REPETIDOS: ninguno no es una buena noticia: significa que la base te salvó. Sin esa restricción —que es el cinturón del paso 4— ahora tendrías once folios duplicados en una tabla legal.

Los números concretos van a cambiar en tu máquina y entre corridas. Puede que te sobrevivan 9 o 12. Lo que **no** cambia es que **no sobreviven 20**: eso es lo que hay que ver.

Vas bien si...

rechazados por la base es un número **mayor que cero**, y folios en la tabla es **menor que 21**. Si te salieran 21 limpios, el turno del paso 4 ya está puesto.

Si te atascas

1 · Salen 21 folios y cero rechazos.

Estás corriendo la demo del paso 4 por error, o le pusiste ya el turno al método ingenuo.

2 · La aplicación revienta entera.

No debería: las excepciones de las emisiones fallidas están capturadas y contadas a propósito. Si la aplicación se cae, mira si tocaste el manejo de errores de la demo.

3.3 Paso 3 · La trampa: *synchronized* no sirve

Qué vamos a hacer

Entender por qué la primera idea que a todo el mundo se le ocurre **no arregla nada**. Este paso es de leer.

Para entenderlo mejor

Poner un guardia **en la puerta de tu oficina** para que entre un funcionario cada vez. Funciona... **mientras solo haya una oficina**. El día que la DGT abra una segunda sede con su propio guardia, los dos dejan pasar a uno cada uno — y los dos van al mismo cajón.

El problema

`synchronized` sincroniza **los hilos de una JVM**. Y en producción casi nunca hay una sola JVM: hay dos o tres instancias detrás de un balanceador, precisamente para aguantar la carga.

La alternativa, y por qué no

- **synchronized**: arregla el problema en tu portátil y desaparece al escalar. **Peor aún**: hace que el problema no se reproduzca en pruebas y sí en producción.
- **Un Lock distribuido** (Redis, Zookeeper): funciona, y trae una pieza de infraestructura más que hay que operar y que puede caerse.
- **Un turno pedido a la base de datos**, que es lo del paso siguiente: la base **ya está compartida por todas las instancias**, así que es el único sitio donde todos coinciden. No hay que instalar nada.
- **Una secuencia de la base** (SEQUENCE): es la respuesta correcta cuando basta con «un número único creciente». Aquí no basta, porque el requisito es **correlativo por año y sin saltos**, y una secuencia deja huecos cuando una transacción falla.

3.4 Paso 4 · El turno con nombre

Qué vamos a hacer

Pedirle a la base **un turno**, con nombre, mientras se calcula el número.

Para entenderlo mejor

El número que se reparte en la puerta de una oficina. No hay ninguna ventanilla cerrada con llave: hay **un turno que se pide**, y el que lo tiene pasa mientras los demás esperan. Cuando termina, lo suelta y entra el siguiente — que ahora sí ve el mostrador actualizado.

Aquí el turno se llama **2026**, que es el año. Y fíjate en lo que **no** hace falta: ninguna fila que represente ese turno. El nombre basta.

El problema

Hay que impedir que dos transacciones lean el mismo máximo. Y la única forma de impedirlo es que la segunda **espere** a que la primera termine.

La alternativa, y por qué no

- **Bloqueo optimista** (@Version): no bloquea; deja que los dos escriban y **el segundo falla**, y tú reintentas. Es mejor cuando los choques son **raros** — le ahorras la espera a todo el mundo. Aquí los choques son la norma, así que reintentar veinte veces sería peor.
- **Bloqueo pesimista sobre una fila** (@Lock(PESSIMISTIC_WRITE)): el primero se lleva la fila y los demás esperan. Es lo correcto cuando **lo que se protege es esa fila** — un saldo, un stock. Aquí no lo es: lo que se protege es el cálculo $\max(\text{numero}) + 1$ sobre toda la tabla,

y para usar este mecanismo habría que inventarse una fila cuyo único trabajo fuera ser bloqueada.

- **Un advisory lock**, que es lo de aquí: un turno **con nombre**, no atado a ninguna fila. PostgreSQL se lo da al primero que lo pide y hace esperar al resto.
- **Una secuencia** (`nextval`): atómica, rapidísima, y **deja huecos** cuando una transacción aborta. Un folio tributario saltado hay que explicarlo.

Se pega

En `practica/src/main/java/cl/dgt/concurrencia/repositories/FolioRepository.java`:

```
@Query(value = "select pg_advisory_xact_lock(:anio)", nativeQuery = true)
Object tomarElTurnoDelAnio(@Param("anio") long anio);
```

Y el método que lo usa, en el servicio:

```
@Transactional
public Folio emitirConTurno(int anio) {
    folios.tomarElTurnoDelAnio(anio);

    if (turnoAnunciado.compareAndSet(false, true)) {
        log.info("[TURN0] pg_advisory_xact_lock({}) · el turno vive en la base, no en
↪ Java", anio);
    }

    int ultimo = folios.maxNumeroDe(anio).orElse(0);
    return folios.save(new Folio(anio, ultimo + 1));
}
```

Compáralo con el del paso 1. La diferencia es **una línea**: la que pide el turno antes de mirar el máximo. (La del `log` es sólo para que lo veas en pantalla una vez.)

El turno dura exactamente lo que dure la transacción, y por eso la función se llama `pg_advisory_xact_**lock**` con ese `xact` en medio: se suelta solo al confirmar o al abortar, sin que nadie tenga que acordarse. Por eso el método es `@Transactional`: sin transacción no hay nada que lo sostenga, y se soltaría al instante.

Lo que vas a ver

```
=== 3 · CON TURNO · 20 a la vez, con un lock con nombre ===
año 2026 reiniciado: solo el folio 2026-0001
[TURN0] pg_advisory_xact_lock(2026) · el turno vive en la base, no en Java
folios en la tabla : 21
números distintos  : 21
REPETIDOS          : ninguno
rechazados por la base : 0
emitidos: [2026-0001, 2026-0002, ... 2026-0021]
```

21 de 21. Cero rechazos. Las mismas veinte peticiones simultáneas que antes perdían la mitad.

Y el SQL, que en este laboratorio está encendido:

```
Hibernate:
  select
    pg_advisory_xact_lock(?)
```

Una línea, ninguna tabla, ningún `for update`. Si buscas `for update` en la consola **no vas a encontrar nada**: no se está bloqueando ninguna fila.

Vas bien si...

folios en la tabla: 21, números distintos: 21 y rechazados por la base: 0. Los tres a la vez.

Si te atascas

1 · Siguen saliendo rechazos.

Dos causas, por frecuencia: falta el `@Transactional` en el método del servicio, o pediste el turno **después** de leer el máximo — el orden es lo único que importa aquí.

2 · La demo se queda colgada y no termina.

Es lo que pasa cuando el turno no se suelta: alguien lo tiene y no cierra su transacción. Corta con `Ctrl+C` y comprueba que el método sea `@Transactional` y termine.

3 · Could not determine recommended JdbcType o un error de tipos en el parámetro.

El parámetro tiene que ser `long`, no `int`: `pg_advisory_xact_lock` tiene una forma que toma un `bigint` y otra que toma dos `int`, y con `long` no hay ambigüedad.

3.5 Paso 5 · El cinturón

Qué vamos a hacer

Mirar la restricción de la base que ya te salvó en el paso 2, y entender por qué se tienen **las dos** cosas.

Para entenderlo mejor

El turno es **el procedimiento**: si todo el mundo lo pide, no hay duplicados. La restricción `UNIQUE` es **la propiedad del talonario**: aunque alguien se salte el procedimiento, el talonario no admite dos hojas iguales.

El problema

El turno protege el código que lo pide. No protege de un método nuevo que alguien escriba el año que viene sin acordarse, ni de una carga de datos hecha a mano, ni de un script de migración.

La alternativa, y por qué no

No son dos opciones entre las que elegir: **son dos capas**, y hacen falta las dos.

- **Sólo la restricción**: la base rechaza los duplicados, sí — y tu aplicación pierde la mitad de las peticiones, como viste en el paso 2. Una restricción no coordina: sólo dice que no.
- **Sólo el turno**: coordina bien, y no protege de quien no pase por ahí.
- **Las dos**: el turno hace que las cosas salgan bien, y la restricción garantiza que, si algo sale mal, **falla en vez de corromper**.

Lo que vas a ver

Vuelve a mirar la salida del paso 2:

```
Detail: Key (anio, numero)=(2026, 4) already exists.
```

Eso fue la restricción trabajando. Sin ella, ese folio duplicado se habría guardado sin que nadie se enterara, y el problema se habría descubierto meses después, en una auditoría.

Vas bien si...

Puedes explicar, con tus palabras, por qué se tienen las dos cosas y qué protege cada una.

4 Lo que aprendiste

1 · Correcto para uno no es correcto para veinte.

El código del paso 1 no tiene ningún error a la vista y perdió la mitad de las peticiones en el paso 2. La concurrencia no se ve leyendo: hay que provocarla.

2 · `synchronized` no sirve para esto.

Sincroniza los hilos de una JVM, y en producción hay varias. Es peor que no hacer nada, porque el problema deja de reproducirse en tu máquina y sigue vivo donde importa.

3 · El turno va en el sitio compartido: la base.

Es lo único que todas las instancias ven igual. Y se pide **con un nombre**, no sobre una fila: la fila que vas a crear todavía no existe, y ninguna otra representa «el derecho a emitir folios de 2026». Un `synchronized` sólo coordina hilos de una JVM; esto coordina procesos y máquinas.

4 · Turno y restricción son dos capas, no dos opciones.

El turno hace que salga bien. La restricción hace que, si sale mal, **falle** en vez de guardar un dato corrupto. En material tributario, esa diferencia es la que importa.

5 Para profundizar

- **Quita la línea del turno** y vuelve a correr la demo 3. ¿Cuántos rechazos salen ahora?
- **Sube las emisiones simultáneas de 20 a 100** en la demo y mira cuánto tarda la versión con turno. Ése es el precio de hacer cola.
- **Quita la restricción `UNIQUE`** de la migración —en una base de pruebas— y corre la demo 2. Cuenta los duplicados que quedan guardados. Es lo que pasa sin cinturón.
- **Investiga `@Version`** y escribe una versión optimista del emisor. ¿Cuántos reintentos hacen falta con veinte a la vez?

6 Antes de cerrar

Párala con `Ctrl+C`.

```
./mvnw clean
```

Lo que te llevas:

Entre leer y escribir hay un hueco, y con varios usuarios a la vez ahí caben los demás. El turno se le pide a la base, tiene nombre propio y dura lo que dure la transacción. Y encima del turno va una restricción, porque el procedimiento sólo protege a quien lo sigue.

Lo que queda pendiente, y abre el Lab 08: todo lo que llevas comprobado lo has comprobado **mirando la consola**. Eso no escala y no se puede repetir. En el Lab 08 se escribe el primer test, y se ve fallar a propósito.